

FIAP – Faculdade de Informática e Administração Paulista

Pós-Graduação em Arquitetura e Desenvolvimento Java

Sistema para Gerenciamento de consulta

utilizando Spring Boot, GraphQL e comunicação assíncrona via RabbitMQ

Aluno

Pâmela de Araújo Albuquerque

Data

Setembro/2026

1. Introdução

1.1 Contexto

O documento descreve, de forma detalhada e técnica, a arquitetura e a implementação do sistema de agendamento de consulta, desenvolvido como solução para o Tech Challenge da Fase 3 do curso de Arquitetura e Desenvolvimento em Java.

O problema proposto consiste em desenvolver um backend simplificado e modular para um ambiente hospitalar, contemplando:

- Agendamento de consultas
- Gerenciamento de histórico de pacientes
- Envio automatizado de lembretes
- Controle de acesso por perfil de usuário (Médicos, Enfermeiros e Pacientes)

1.2 Escopo

A solução implementada adota uma arquitetura de microsserviços, dividida em dois serviços independentes que se comunicam de forma assíncrona por meio de um *message broker* (RabbitMQ). A camada de exposição de dados utiliza GraphQL para consultas flexíveis e REST para operações administrativas (cadastro e autenticação).

1.3 Motivação

A escolha por esse conjunto de tecnologias se justifica pela necessidade de:

1. **Desacoplamento** entre os domínios de agendamento e notificação;
2. **Flexibilidade** na consulta de dados médicos, cuja estrutura é complexa e variável;
3. **Segurança** robusta, com autenticação e autorização em múltiplas camadas;
4. **Escalabilidade**, uma vez que cada serviço pode ser escalado independentemente;
5. **Resiliência**, dado que falhas em um serviço não comprometem o outro.

2. Objetivos do Projeto

2.1 Objetivo Geral

Desenvolver um backend modular, seguro e escalável para gestão de agendamento de consultas, utilizando boas práticas de arquitetura de microsserviços e comunicação assíncrona.

2.2 Objetivos Específicos

	Objetivo	Resultado Esperado
1	Implementar autenticação básica com Spring Security	Acesso controlado por perfil
2	Modelar três perfis de usuário	Médico, Enfermeiro, Paciente

3	Disponibilizar consultas flexíveis via GraphQL	Filtros de consultas completo e futuro
4	Separar o sistema em dois serviços independentes	Agendamento e Notificação
5	Implementar comunicação assíncrona com RabbitMQ	Producer/Consumer de eventos
6	Containerizar a solução completa	docker-compose funcional

3. Fundamentação Teórica

Esta seção apresenta os conceitos teóricos que embasam as decisões técnicas adotadas.

3.1 Arquitetura de Microsserviços

Microsserviços consistem em uma abordagem arquitetural na qual a aplicação é decomposta em serviços pequenos, independentes e fracamente acoplados, cada um responsável por um contexto de negócio específico. Segundo Fowler e Lewis (2014), esse estilo arquitetural apresenta como principais benefícios:

- **Independência tecnológica** entre serviços;
- **Escalabilidade horizontal** por componente;
- **Deploy independente**;
- **Tolerância a falhas** (isolamento de falhas).

No presente projeto, essa abordagem é aplicada separando os domínios de agendamento e notificação em serviços distintos.

3.2 GraphQL

GraphQL é uma linguagem de consulta para APIs, criada pelo Facebook em 2012 e posteriormente aberta à comunidade. Diferentemente do modelo REST, no qual cada recurso é exposto por um endpoint específico, o GraphQL concentra toda a API em um único endpoint, permitindo que o cliente especifique exatamente quais campos deseja receber.

Vantagens relevantes para este projeto:

Vantagem	Descrição
Sem overfetching	O cliente solicita apenas os campos necessários
Sem underfetching	Uma única query pode buscar dados relacionados
Tipagem forte	O schema atua como contrato entre cliente e servidor
Evolução contínua	Novos campos podem ser adicionados sem quebrar clientes existentes

3.3 Mensageria e Comunicação Assíncrona

A comunicação assíncrona via *message broker* é um padrão amplamente utilizado em sistemas distribuídos para desacoplar produtores e consumidores de mensagens. No presente projeto, utiliza-se o RabbitMQ, que implementa o protocolo AMQP 0-9-1.

Benefícios:

1. Desacoplamento temporal — produtor e consumidor não precisam estar ativos simultaneamente;

2. Resiliência — mensagens podem ser persistidas e reprocessadas;
3. Escalabilidade — múltiplos consumidores podem processar a mesma fila;
4. Flexibilidade de roteamento — exchanges permitem roteamento baseado em chaves.

3.4 Spring Security e Autenticação Básica

O Spring Security é um framework de autenticação e autorização que implementa uma cadeia de filtros (*filter chain*) para interceptar requisições HTTP. A autenticação básica (Basic Auth) é um esquema definido pela RFC 7617, no qual as credenciais são enviadas em um header HTTP codificado em Base64.

Este projeto utiliza Basic Auth por ser:

- **Simple** e adequado ao escopo acadêmico;
- **Nativo** do Spring Security;
- **Aderente** ao requisito explícito do enunciado.

3.5 Docker e Containerização

Docker é uma plataforma de virtualização em nível de sistema operacional que permite empacotar aplicações e suas dependências em containers. Cada container é isolado, mas compartilha o kernel do host, sendo muito mais leve que uma máquina virtual.

O Docker Compose, por sua vez, permite orquestrar múltiplos containers por meio de um arquivo YAML declarativo, viabilizando um ambiente reproduzível.

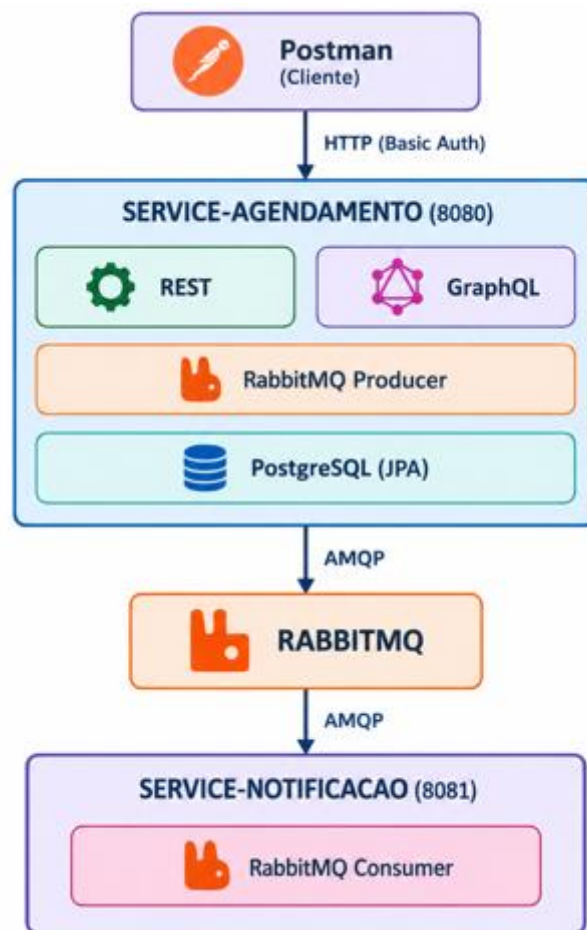
4. Arquitetura da Solução

4.1 Visão Geral

A solução é composta por quatro componentes principais, todos executando em containers Docker:

Componente	Porta	Responsabilidade
service-agendamento	8080	REST (auth) + GraphQL + RabbitMQ Producer
service-notificacao	8081	RabbitMQ Consumer
postgres	5432	Persistência relacional
rabbitmq	5672 / 15672	Broker de mensagens

4.2 Diagrama C4 — Nível de Contexto



4.3 Fluxo de uma Operação Completa

Cenário: Médico cria uma consulta para um paciente.

1. **Cliente** envia mutation GraphQL para /graphql com credenciais Basic Auth;
2. **Spring Security** autentica o usuário e verifica ROLE_MEDICO;
3. **ConsultaResolver** invoca ConsultaService.criarConsulta();
4. **ConsultaService** valida o paciente, o médico e persiste no PostgreSQL;
5. **ConsultaResolver** cria um EventoNotificacaoDTO;
6. **ConsultaProducer** publica o evento na exchange consulta.exchange com routing key consulta.criada;
7. **RabbitMQ** encaminha a mensagem para a fila notificacao.queue;
8. **NotificacaoConsumer** (do outro serviço) recebe a mensagem;
9. **NotificacaoConsumer** loga o lembrete para o paciente.

5. Modelagem de Domínio

5.1 Entidades

5.1.1 Usuario

Representa qualquer usuário do sistema. Como todos os perfis compartilham o mesmo conjunto de atributos (com exceção de campos específicos como crm e coren, que não foram exigidos), optou-se por uma **modelagem única** com um discriminador de perfil.

Atributo	Tipo	Descrição	Restrições
id	Long	Identificador	PK, auto-increment
username	String	Nome de usuário	Único, não nulo
password	String	Senha criptografada	BCrypt, não nulo
enabled	Boolean	Conta ativa	Padrão: true
nome	String	Nome completo	—
email	String	Email	Único
role	Enum	Perfil	MEDICO, ENFERMEIRO, PACIENTE

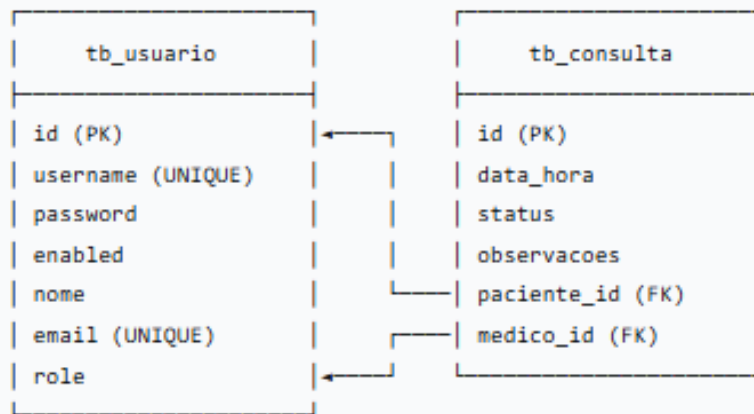
Implementação: a classe Usuario implementa a interface UserDetails do Spring Security, o que permite sua integração direta ao mecanismo de autenticação.

5.1.2 Consulta

Representa uma consulta médica agendada.

Atributo	Tipo	Descrição	Restrições
id	Long	Identificador	PK
dataHora	LocalDateTime	Data e hora	Não nulo
status	Enum	Estado da consulta	AGENDADA, REALIZADA, CANCELADA
observacoes	String	Anotações do médico	—
paciente	Usuario	Referência ao paciente	@ManyToOne
medico	Usuario	Referência ao médico	@ManyToOne

5.2 Diagrama Entidade-Relacionamento



5.3 Estratégias de Modelagem Adotadas

Decisão 1: Modelo único de usuário com enum Role

Justificativa: os perfis Médico, Enfermeiro e Paciente compartilham o mesmo conjunto de atributos no escopo do problema. A criação de tabelas separadas (herança JPA) aumentaria a complexidade sem benefícios proporcionais. Além disso, o discriminador role é suficiente para direcionar o controle de acesso, atendendo ao requisito de forma direta.

Decisão 2: Uso de @ManyToOne para paciente e médico

Justificativa: um paciente pode ter várias consultas, mas cada consulta tem um único paciente. O mesmo vale para médico. O uso de @ManyToOne reflete corretamente essa cardinalidade.

Decisão 3: Uso de Enum para status e role

Justificativa: ambos são domínios fechados, com valores bem definidos. O uso de @Enumerated(EnumType.STRING) garante legibilidade no banco e robustez a mudanças de ordem no enum.

6. Detalhamento dos Microserviços

6.1 Service-Agendamento

6.1.1 Responsabilidades

- Expor endpoints REST para cadastro e listagem de usuários;
- Expor API GraphQL para operações sobre consultas;
- Persistir dados no PostgreSQL;
- Publicar eventos no RabbitMQ.

6.1.2 Estrutura de Pacotes

br.com.service.agendamento	
└ ServiceAgendamentoApplication.java	(Bootstrap)
└ config/	
└ SecurityConfig.java	(Spring Security)
└ RabbitMQConfig.java	(RabbitMQ: exchange, fila, bindings)
└ controller/	
└ UsuarioController.java	(REST: cadastro, listagem)
└ resolver/	
└ ConsultaResolver.java	(GraphQL: queries e mutations)
└ service/	
└ ConsultaService.java	(Regras de negócio)
└ UsuarioService.java	(Cadastro e consultas de usuários)
└ repository/	
└ ConsultaRepository.java	(JPA)
└ UsuarioRepository.java	(JPA)
└ entity/	
└ Consulta.java	(Entidade JPA)
└ Usuario.java	(Entidade JPA + UserDetails)
└ dto/	
└ ConsultaInput.java	(record – entrada GraphQL)
└ EventoNotificacaoDTO.java	(record – mensagem AMQP)
└ UsuarioRequest.java	(record – cadastro REST)
└ UsuarioResponse.java	(record – resposta REST)
└ producer/	
└ ConsultaProducer.java	(Publicação de eventos)
└ security/	
└ CustomUserDetailsService.java	(Integração JPA + Spring Security)
└ exception/	
└ BusinessException.java	
└ GlobalExceptionHandler.java	
└ GraphQLExceptionHandler.java	
└ ResourceNotFoundException.java	

6.1.3 Padrões Aplicados

Padrão	Aplicação
Resolver Pattern	ConsultaResolver como camada de exposição GraphQL
Repository Pattern	Interfaces JpaRepository para acesso a dados
DTO	record para transferência de dados entre camadas
Producer/Consumer	ConsultaProducer publica eventos
Dependency Injection	Injeção via construtor (Lombok @RequiredArgsConstructor)

6.2 Service-Notificacao

6.2.1 Responsabilidades

- Consumir mensagens da fila notificacao.queue;
- Simular o envio de lembretes (log).

6.2.2 Estrutura de Pacotes

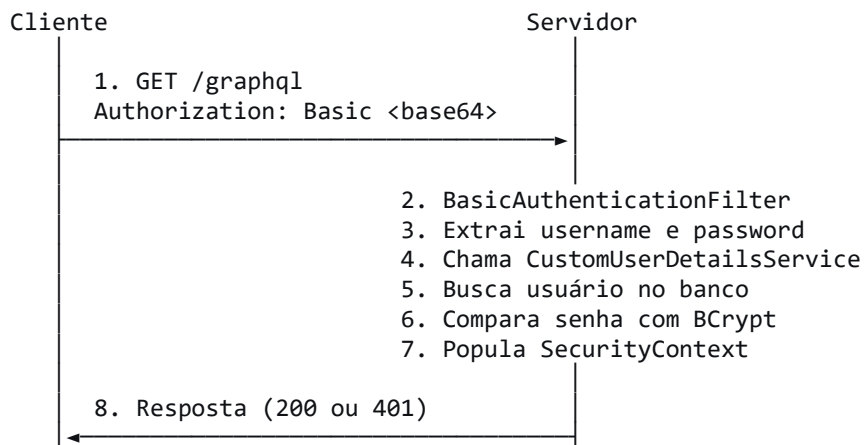
br.com.service.notificacao	
├ ServiceNotificacaoApplication.java	(Bootstrap)
├ config/	
│ └ RabbitMQConfig.java	(Declaração da fila e exchange)
├ consumer/	
│ └ NotificacaoConsumer.java	(Escuta a fila)
└ dto/	
└ EventoNotificacaoDTO.java	(record – mensagem AMQP)

7. Segurança e Controle de Acesso

7.1 Modelo de Segurança

O projeto adota autenticação básica (Basic Auth) aliada a controle de acesso baseado em roles (RBAC).

7.2 Fluxo de Autenticação



7.3 Configuração do SecurityConfig

Os pontos-chave da configuração são:

Configuração	Valor	Justificativa
csrf	desabilitado	API stateless
sessionCreationPolicy	STATELESS	Não há sessão HTTP
/api/auth/cadastrar	permitAll	Cadastro deve ser público
/graphql	authenticated	Requer autenticação
anyRequest	authenticated	Restante requer autenticação
httpBasic	habilitado	Esquema escolhido

7.4 Matriz de Autorização

Operação	Médico	Enfermeiro	Paciente
Cadastrar usuário	☒	☒	☒
Listar usuários	☒	☒	☒
Listar usuários por perfil	☒	☒	☒
Listar todas as consultas	☒	☒	☒ (apenas as suas)
Listar consultas futuras	☒	☒	☒ (apenas as suas)
Consultar por ID	☒	☒	☒ (apenas se for sua)
Criar consulta	☒	☒	☒
Editar consulta	☒	☒	☒
Alterar status da consulta	☒	☒	☒
Cancelar consulta	☒	☒	☒

7.5 Mecanismo de Autorização Implementado

A autorização é aplicada em duas camadas:

Camada 1 — Declarativa (@PreAuthorize):

```
@PreAuthorize("hasAnyRole('MEDICO', 'ENFERMEIRO')")
public Consulta criarConsulta(@Argument ConsultaInput input) { ... }
```

Camada 2 — Programática (ConsultaService):

```
public List<Consulta> listarConsultas(Usuario usuarioLogado) {
    if (usuarioLogado.getRole() == Usuario.Role.PACIENTE) {
        return consultaRepository.findByPaciente(usuarioLogado);
    }
    return consultaRepository.findAll();
}
```

A camada declarativa bloqueia perfis sem permissão; a programática garante que pacientes só vejam seus próprios dados (filtro fino).

7.6 Criptografia de Senhas

As senhas são armazenadas com BCrypt, um algoritmo de *hashing* adaptativo que:

1. Incorpora um salt aleatório por senha;
2. Possui um fator de custo configurável (padrão: 10);
3. É resistente a ataques de força bruta e tabelas pré-computadas (*rainbow tables*).

A configuração é feita por meio do bean PasswordEncoder.

8. API GraphQL

8.1 Schema

O schema GraphQL é o contrato entre cliente e servidor. É definido em `src/main/resources/graphql/schema.graphqls`.

Types:

- Consulta — Representa uma consulta médica
- Usuario — Representa um usuário (médico, enfermeiro ou paciente)
- ConsultaInput — Objeto de entrada para mutações

Queries:

- consultas — Lista todas as consultas (filtradas por perfil)
- consultasFuturas — Lista apenas consultas futuras
- consulta(id) — Consulta específica por ID

Mutations:

- `criarConsulta(input)` — Cria nova consulta
- `editarConsulta(id, input)` — Edita consulta existente
- `cancelarConsulta(id)` — Cancela consulta
- `RealizarConsulta` — Altera status da consulta para realizada

8.2 Boas Práticas Aplicadas

Prática	Implementação
Nomes no singular/plural	<code>consulta</code> (singular), <code>consultas</code> (plural)
Uso de input para mutações	<code>ConsultaInput</code> em vez de parâmetros soltos
Tipagem forte	Todos os campos têm tipo definido
Nulidade explícita	<code>ID!</code> , <code>String!</code> (não-nulos)

8.3 Tratamento de Erros

Erros são capturados pelo `GraphQLExceptionHandler`, que implementa `DataFetcherExceptionHandlerAdapter`. Mensagens de `BusinessException` e `ResourceNotFoundException` são retornadas ao cliente no formato padrão do GraphQL:

```
{
  "errors": [
    {
      "message": "Não é possível editar uma consulta já realizada."
    }
  ]
}
```

8.4 Segurança

Cada método do `ConsultaResolver` é protegido por `@PreAuthorize`, garantindo que o Spring Security valide a autorização antes mesmo da execução da lógica de negócio.

9. Comunicação Assíncrona

9.1 Modelo de Mensageria

O projeto utiliza o modelo `publish/subscribe` (pub/sub) sobre uma `Topic Exchange`:

Recurso	Nome
Exchange	consulta.exchange
Type	Topic
Queue	notificacao.queue
Routing Key (criação)	consulta.criada
Routing Key (edição)	consulta.editada

9.2 Payload da Mensagem

A mensagem é um objeto JSON serializado a partir do record EventoNotificacaoDTO:

```
{
  "consultaId": 1,
  "pacienteEmail": "joao@email.com",
  "pacienteNome": "João Silva",
  "dataHora": "2026-09-10T14:30:00",
  "acao": "CRIADA"
}
```

9.3 Fluxo Completo

1. ConsultaResolver.criarConsulta()
- ↓
2. ConsultaService.criarConsulta() persiste no banco
- ↓
3. ConsultaResolver cria EventoNotificacaoDTO
- ↓
4. ConsultaProducer.enviarEventoCriacao()
- ↓
5. RabbitTemplate.convertAndSend(exchange, routingKey, evento)
- ↓
6. RabbitMQ roteia para notificacao.queue
- ↓
7. NotificacaoConsumer.receberEvento() é acionado
- ↓
8. Lembrete é logado

9.3 Configuração do RabbitMQConfig

Cada serviço possui sua própria classe RabbitMQConfig, porém ambas declaram os mesmos recursos (exchange, fila, bindings), garantindo idempotência. Essa é uma prática recomendada no RabbitMQ, uma vez que a declaração é idempotente: se o recurso já existir, a operação é ignorada.

9.4 Uso de record e Jackson2JsonMessageConverter

A serialização das mensagens é realizada por meio do Jackson2JsonMessageConverter, que converte automaticamente objetos Java em JSON e vice-versa. O uso de record (Java 21) elimina boilerplate e garante imutabilidade das mensagens.

10. Infraestrutura e Deploy

10.1 Containerização

Cada serviço possui seu próprio Dockerfile, seguindo o padrão multi-stage build:

- **Estágio 1 (build):** utiliza maven:3.9.6-eclipse-temurin-21 para compilar o projeto;
- **Estágio 2 (runtime):** utiliza eclipse-temurin:21-jre-alpine, mais leve, apenas com o JRE.

Essa abordagem resulta em imagens menores e mais seguras.

10.2 Orquestração com Docker Compose

O arquivo docker-compose.yml declara quatro serviços:

Serviço	Imagem	Porta(s)
postgres	postgres:16-alpine	5432
rabbitmq	rabbitmq:3.13-management-alpine	5672, 15672
service-agendamento	Build local	8080
service-notificacao	Build local	8081

10.3 Rede e Volumes

- **Rede:** tech-network (bridge), permitindo comunicação entre containers por nome.
- **Volumes:** postgres_data, garantindo persistência dos dados.

11. Testes e Validação

11.1 Estratégia de Testes

Os testes foram conduzidos de forma manual, cobrindo:

1. **Testes de cadastro** (sucesso e falhas esperadas);
2. **Testes de autenticação** (Basic Auth);

3. **Testes de autorização** (por perfil);
4. **Testes de GraphQL** (queries e mutations);
5. **Testes de mensageria** (via logs).

11.2 Cenários Cobertos

#	Cenário	Resultado Esperado
1	Cadastro com username único	201 Created
2	Cadastro com username duplicado	400 Bad Request
3	Cadastro com email duplicado	400 Bad Request
4	Cadastro com role inválida	400 Bad Request
5	Requisição sem autenticação	401 Unauthorized
6	Paciente tentando criar consulta	403 Forbidden
7	Médico criando consulta	201 + evento publicado
8	Paciente listando suas consultas	200 (apenas as suas)
9	Médico listando todas as consultas	200 (todas)
10	Consumidor recebendo mensagem	Log no console

11.2.1 COLLECTION 1 — REST (Cadastro de Usuários e listagem)

A requisição `/api/auth/cadastrar` é pública, e realiza o cadastro de usuários.

Seguir ordem de criação conforme listado abaixo para que as consultas feitas na COLLECTION 2 - GraphQL funcionem sem ser necessário preencher os dados.

Criar os usuários na ordem:

- Médico
- Paciente 1

- Paciente 2
- Enfermeiro

Endpoint: <http://localhost:8080/api/auth/cadastrar>

Descrição: Criar usuário

The screenshot shows a REST client interface for a "Tech Challenge - REST" project. The selected endpoint is "http://localhost:8080/api/auth/cadastrar" with a "POST" method. The request body is raw JSON, containing user details for a doctor named Dr. João Silva. The response status is "201 Created", indicating a successful registration. The response body is JSON, showing the created user's ID, username, password, name, email, role, and various status flags like "enabled", "credentialsNonExpired", "accountNonExpired", "accountNonLocked", and "authorities".

```
1 {
2   "username": "dr_joao",
3   "password": "123456",
4   "nome": "Dr. João Silva",
5   "email": "joao@hospital.com",
6   "role": "MEDICO"
7 }
```

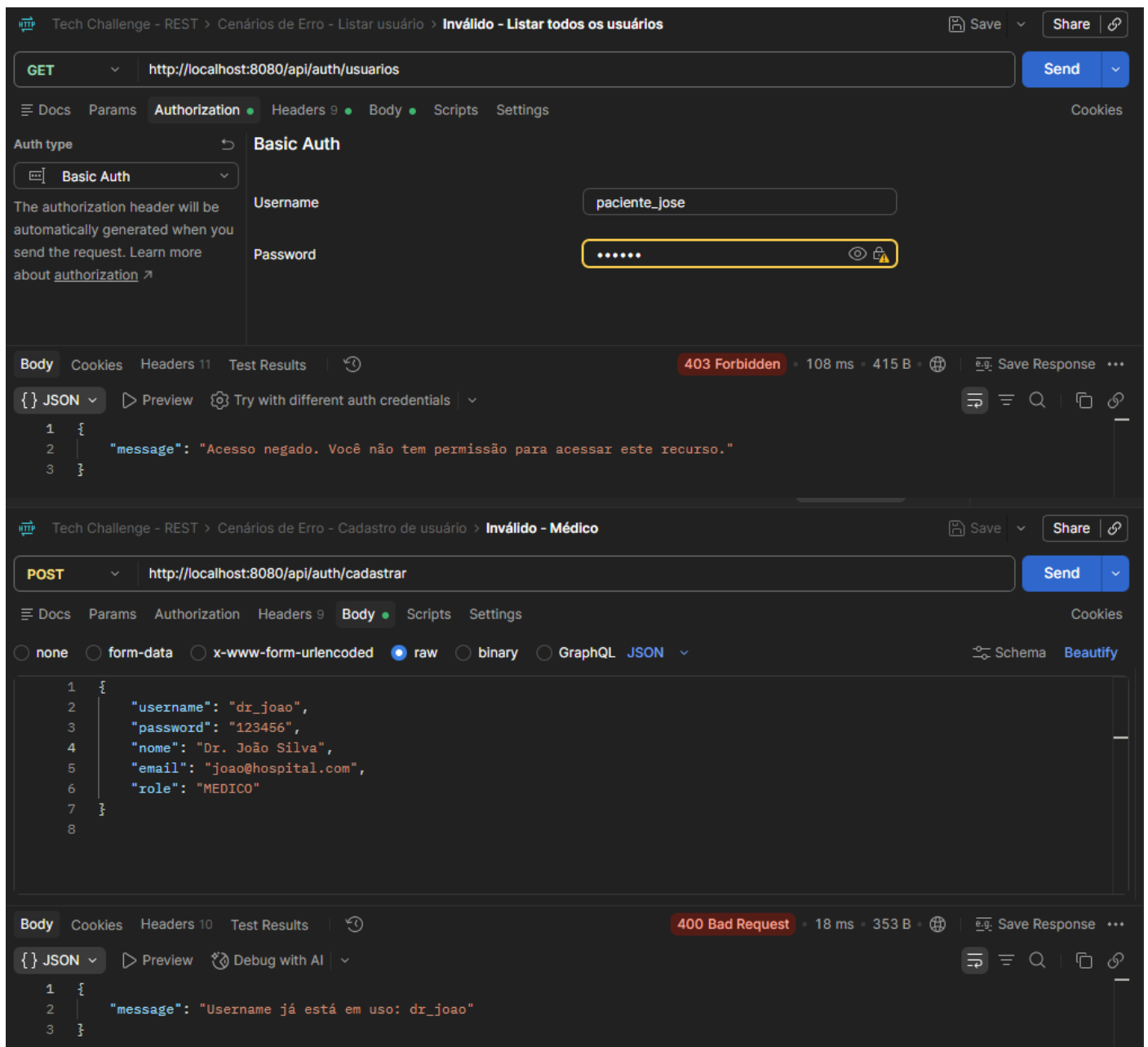
```
1 {
2   "id": 1,
3   "username": "dr_joao",
4   "password": "$2a$10$q5sxf4wnFN/W9sbEyeJFYe.QHbuP.oub68nDrBe.4P8dnoesg0RW2",
5   "enabled": true,
6   "nome": "Dr. João Silva",
7   "email": "joao@hospital.com",
8   "role": "MEDICO",
9   "credentialsNonExpired": true,
10  "accountNonExpired": true,
11  "accountNonLocked": true,
12  "authorities": [
13    "ROLE_MEDICO"
14  ]
15 }
```

Cenários de erros:

The screenshot shows the same REST client interface, but for an error scenario. The endpoint is still "http://localhost:8080/api/auth/cadastrar" with a "POST" method. The request body is raw JSON, attempting to create a patient user named José Oliveira with the email "jose@email.com". The response status is "400 Bad Request", indicating an error. The response body is JSON, with a "message" field stating "Email já está cadastrado: jose@email.com", confirming that the email is already in use.

```
1 {
2   "username": "paciente01",
3   "password": "123456",
4   "nome": "José Oliveira",
5   "email": "jose@email.com",
6   "role": "PACIENTE"
7 }
```

```
1 {
2   "message": "Email já está cadastrado: jose@email.com"
3 }
```

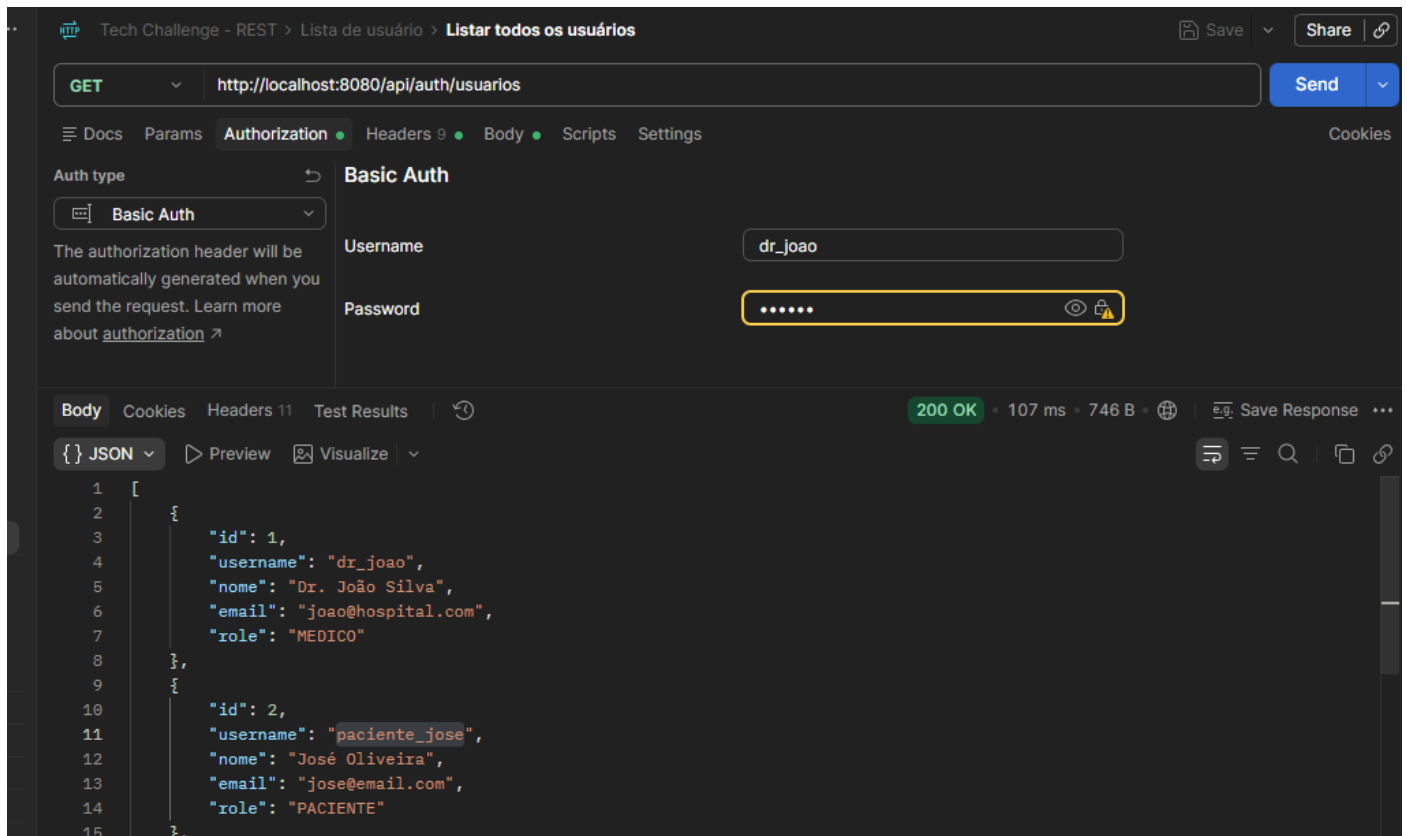


Nas requisições abaixo necessita configurar Basic Auth.

Endpoint: `http://localhost:8080/api/auth/usuarios`

Descrição: Listar Usuários (somente os perfis Medico e Enfermeiro tem permissão).

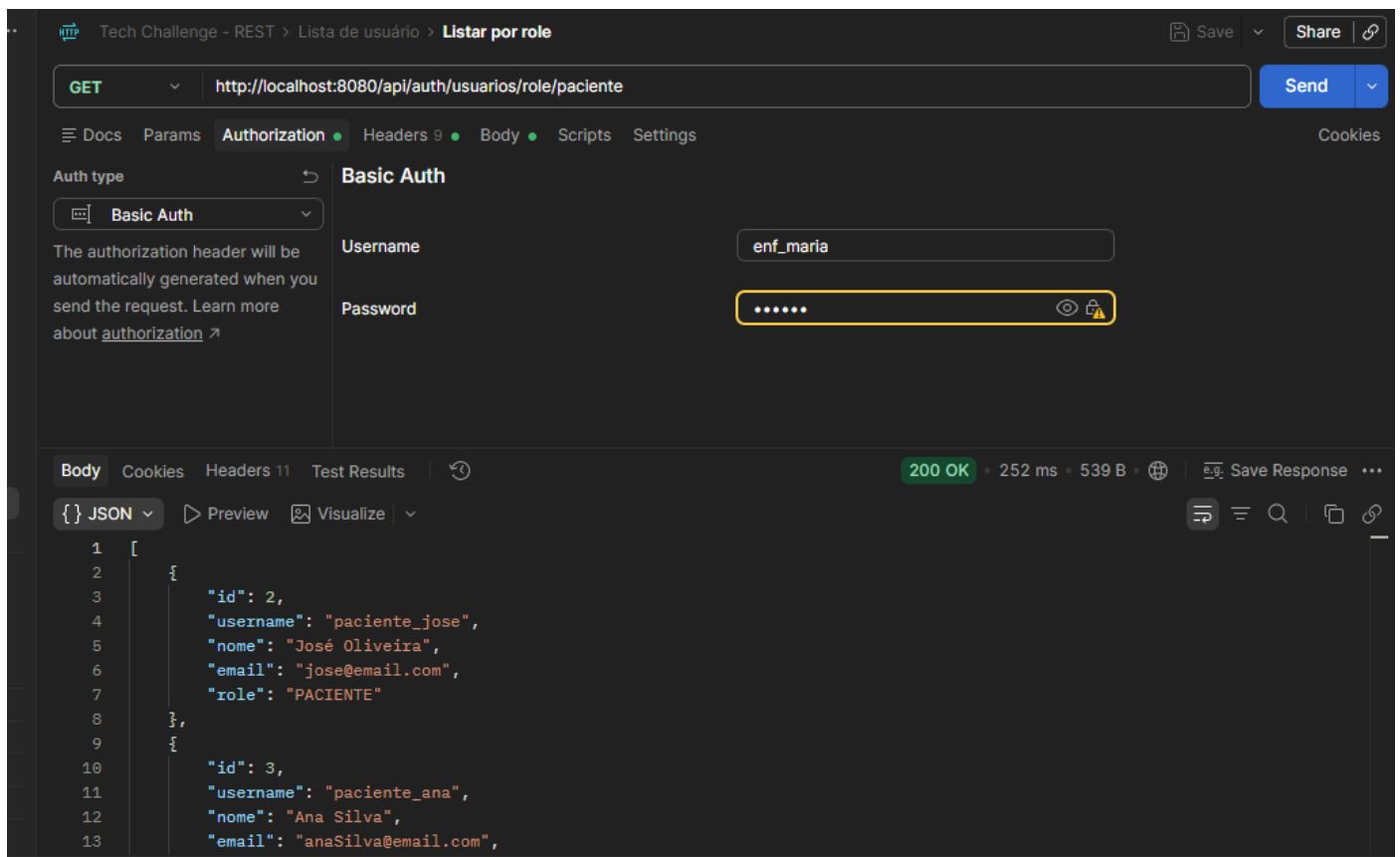
- **Authorization** → **Basic Auth**
- **Username:** `dr_joao`
- **Password:** `123456`



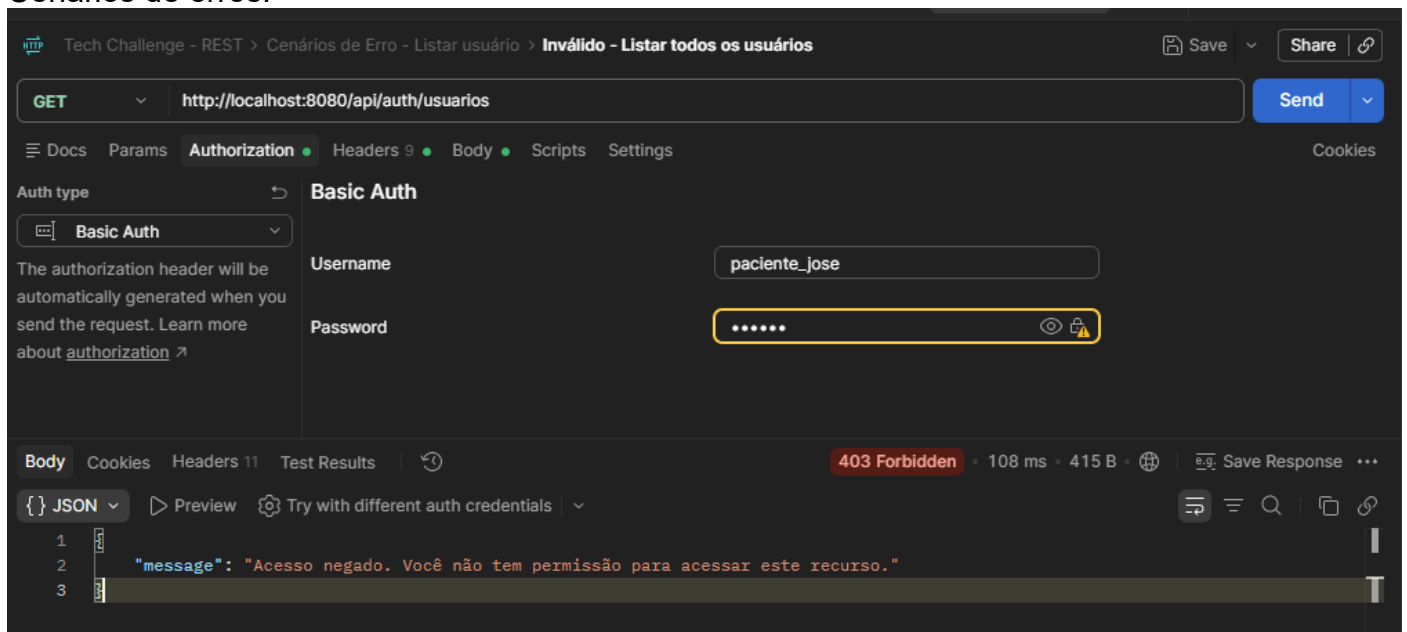
Endpoint: `http://localhost:8080/api/auth/usuarios/role/paciente`

Descrição: Listar por role (os perfis Medico e Enfermeiro tem permissão)

- **Authorization** → **Basic Auth**
- **Username:** `enf_maria`
- **Password:** `123456`



Cenários de erros:



11.2.2 COLLECTION 2 - GraphQL (Consultas)

Para facilitar a validação das funcionalidades da API GraphQL, foram disponibilizadas duas abordagens complementares de teste no Postman, permitindo que o avaliador escolha a que melhor se adequa ao seu fluxo de trabalho:

- **Collection Postman com Operações Pré-configuradas**

A collection Tech Challenge - GraphQL contém todas as queries e mutations necessárias para testar o sistema, já organizadas em grupos lógicos:

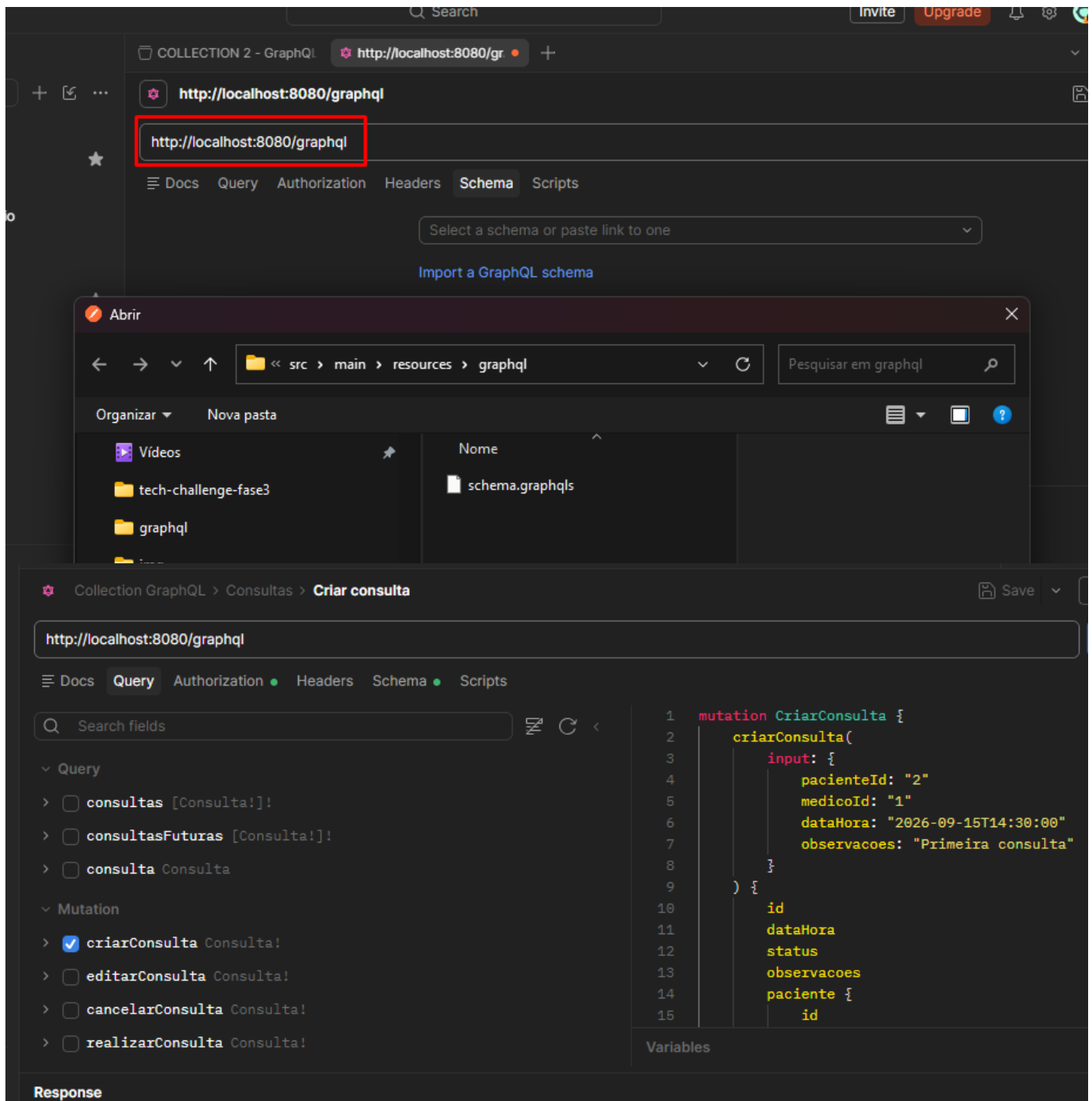
- **Queries (Médico/Enfermeiro):** listar todas as consultas, listar consultas futuras, buscar consulta por ID;
- **Mutations (Médico/Enfermeiro):** criar, editar, cancelar e realizar consultas;
- **Cenários de Paciente:** listagem restrita às suas próprias consultas;
- **Cenários de Erro:** validação de bloqueios por perfil, recursos inexistentes e regras de negócio.

Cada requisição já vem com o Basic Auth configurado para o perfil adequado (médico, enfermeiro ou paciente) e com as variáveis pré-preenchidas, permitindo que o avaliador apenas clique em "Send" para visualizar o resultado esperado.

- **GraphQL Nativo do Postman (via Schema Introspection)**

Alternativamente, o avaliador pode explorar a API de forma interativa por meio do recurso GraphQL nativo do Postman. Para isso, basta:

1. Criar uma nova requisição do tipo GraphQL;
2. Configurar a URL <http://localhost:8080/graphql> e a autenticação Basic Auth;
3. Clicar em "Schema" → "Import Schema" para carregar o schema via introspection;
4. A partir daí, o Postman oferece autocomplete, validação de sintaxe e uma aba dedicada de variáveis, permitindo escrever queries personalizadas e explorar a documentação do schema em tempo real.



- **Equivalência das Abordagens**

Ambas as abordagens são funcionalmente equivalentes — testam os mesmos endpoints e respeitam as mesmas regras de autorização. A escolha entre uma e outra é uma decisão de fluxo de trabalho:

Abordagem	Ideal para
Collection pré-configurada	Validação rápida e reproduzível de todos os cenários

GraphQL nativo (introspection)

Exploração interativa, autocomplete e escrita de novas queries

Adicionalmente, o sistema oferece a interface **GraphiQL** em <http://localhost:8080/graphiql>, que permite testes exploratórios diretamente pelo navegador, sem necessidade de ferramentas externas.

Queries e Mutations

Nas requisições abaixo necessita configurar Basic Auth.

Criar Consulta

Descrição: Cria consultas (os perfis Medico e Enfermeiro tem permissão)

The screenshot shows the GraphiQL web interface. The URL bar displays `http://localhost:8080/graphiql`. The 'Mutation' tab is selected, and a list of mutations is shown on the left, with `criarConsulta` checked. The main editor contains the following GraphQL mutation:

```
1 mutation CriarConsulta {
2   criarConsulta(
3     input: {
4       pacienteId: "2"
5       medicoId: "1"
6       dataHora: "2026-09-15T14:30:00"
7       observacoes: "Primeira consulta"
8     }
9   )
10 }
```

The response is displayed in the 'Body' tab, showing a 200 OK status with a response time of 497.73 ms and a size of 679 B. The JSON response is:

```
{
  "data": {
    "criarConsulta": {
      "id": "1",
      "dataHora": "2026-09-15T14:30",
      "status": "AGENDADA",
      "observacoes": "Primeira consulta",
      "paciente": {
        "id": "2",

```

Logs:

The screenshot displays a series of application logs. Key entries include:

- `agendamento-app` | 2026-09-12T23:19:51.562Z | INFO | 1 --- [service-agendamento] [nio-8080-exec-9] b.c.s.a.service.ConsultaService : Criando nova consult
- `agendamento-app` | 2026-09-12T23:19:51.597Z | INFO | 1 --- [service-agendamento] [nio-8080-exec-9] b.c.s.a.producer.ConsultaProducer : Publicando evento
- `agendamento-app` | 2026-09-12T23:19:51.608Z | INFO | 1 --- [service-agendamento] [nio-8080-exec-9] o.s.a.r.c.CachingConnectionFactory : Attempting to connec
- `rabbitmq-tech-challenge` | 2026-09-12 23:19:51.638415+00:00 | info | <0.776.0> accepting AMQP connection <0.776.0> (172.18.0.5:55462 -> 172.18.0.2:5672)
- `rabbitmq-tech-challenge` | 2026-09-12 23:19:51.683907+00:00 | info | <0.776.0> connection <0.776.0> (172.18.0.5:55462 -> 172.18.0.2:5672) has a client-provided name: rab
- `rabbitmq-tech-challenge` | 2026-09-12 23:19:51.691491+00:00 | info | <0.776.0> connection <0.776.0> (172.18.0.5:55462 -> 172.18.0.2:5672 - rabbitConnectionFactory#6de2344a:0) user 'guest' authenticated and granted access to vhost '/'
- `agendamento-app` | 2026-09-12T23:19:51.696Z | INFO | 1 --- [service-agendamento] [nio-8080-exec-9] o.s.a.r.c.CachingConnectionFactory : Created new connecti
- `agendamento-app` | 2026-09-12T23:19:51.759Z | INFO | 1 --- [service-agendamento] [nio-8080-exec-9] b.c.s.a.producer.ConsultaProducer : Evento publicado c
- `notificacao-app` | 2026-09-12T23:19:51.909Z | INFO | 1 --- [service-notificacao] [ntContainer#0-4] b.c.s.n.consumer.NotificacaoConsumer : Evento recebido |
- `notificacao-app` | 2026-09-12T23:19:51.913Z | INFO | 1 --- [service-notificacao] [ntContainer#0-4] b.c.s.n.consumer.NotificacaoConsumer :

A modal window titled "LEMBRETE DE CONSULTA - CRIADA" is displayed, showing the following details:

- Paciente : José Oliveira
- E-mail : jose@email.com
- Médico : Dr. João Silva
- Data : 15/09/2026 às 14:30
- Status : AGENDADA

Cenário de erro:

The screenshot shows the GraphQL Playground interface. At the top, the URL is `http://localhost:8080/graphql`. The 'Authorization' tab is selected, showing 'Basic Auth' configuration. The username is `paciente_jose` and the password is masked with dots. Below the configuration, the 'Body' tab shows the response in 'Pretty' format. The response is a 200 OK status with a response time of 170.84 ms and a size of 543 B. The JSON response indicates an authorization error: `"data": null`, `"errors": [{"message": "Acesso negado. Você não tem permissão para executar esta ação.", "path": ["criarConsulta"]}]`, and `"locations": []`.

GraphQL > Inválido - Criar consulta

Save Share

`http://localhost:8080/graphql` Query

Docs Query Authorization Headers Schema Scripts Cookies

Auth type Basic Auth

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#)

Basic Auth

Username `paciente_jose`

Password `.....`

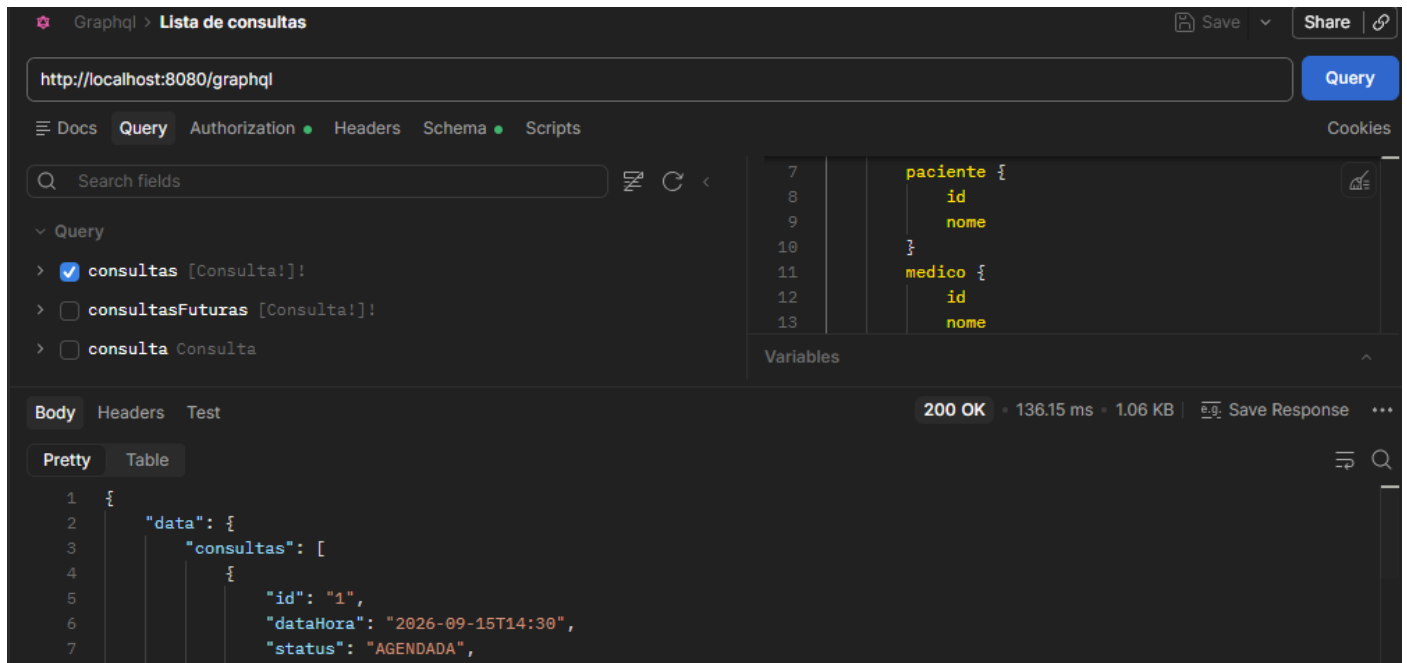
Body Headers Test 200 OK • 170.84 ms • 543 B Save Response

Pretty Table

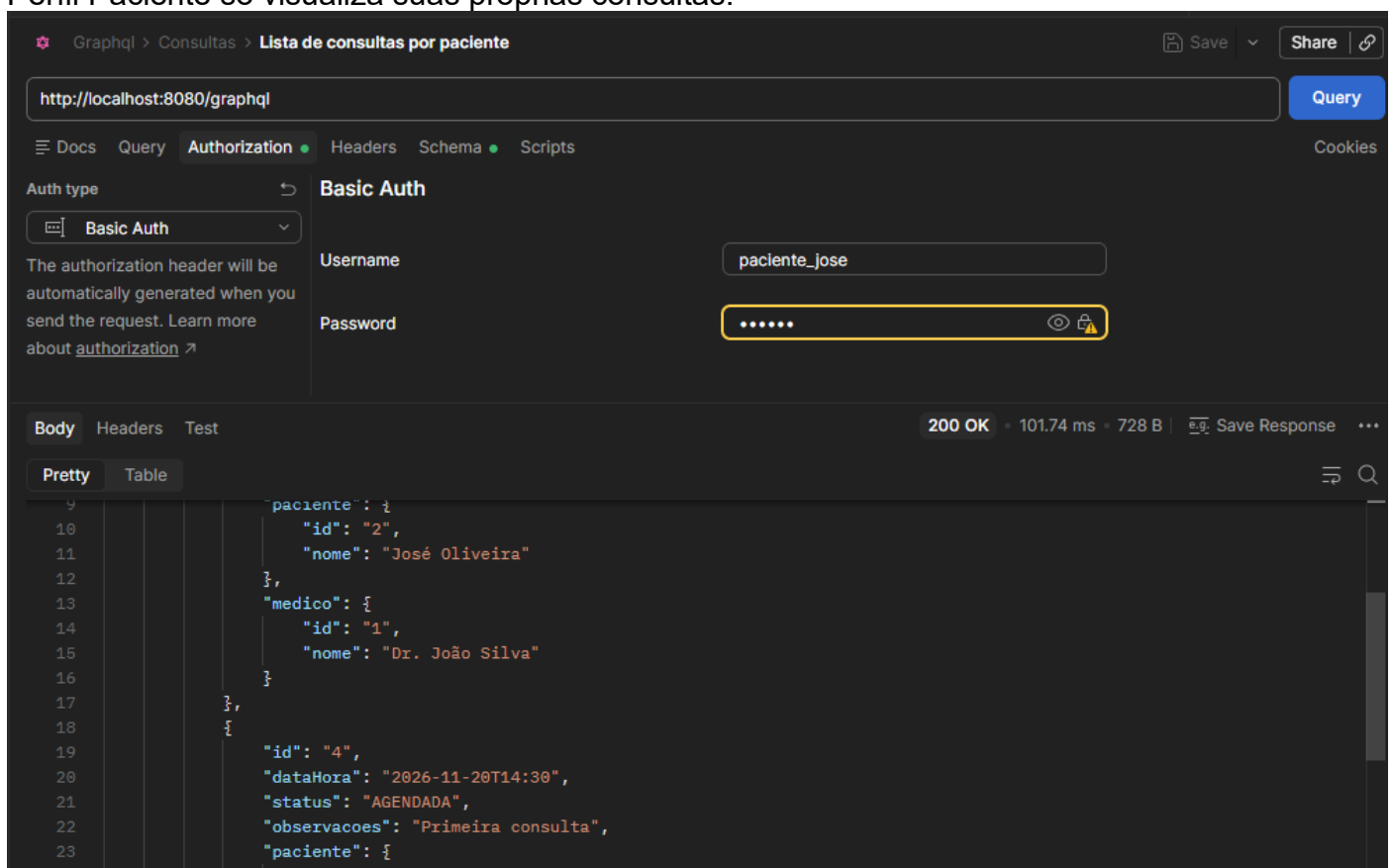
```
1 {
2   "data": null,
3   "errors": [
4     {
5       "message": "Acesso negado. Você não tem permissão para executar esta ação.",
6       "path": [
7         "criarConsulta"
8       ],
9       "locations": [
```

Listar todas as consultas

Descrição: Os perfis Medico e Enfermeiro tem permissão para visualizar todas as consultas.



Perfil Paciente só visualiza suas próprias consultas.



Listar Consultas Futuras

Descrição: Os perfis Medico e Enfermeiro tem permissão para visualizar todas as consultas.

GraphQL > Lista de consultas

http://localhost:8080/graphql

Query

Docs Query Authorization Headers Schema Scripts Cookies

Search fields

Query

- ☒ consultas [Consulta!]!
- ☐ consultasFuturas [Consulta!]!
- ☐ consulta Consulta

Variables

Body Headers Test

200 OK • 136.15 ms • 1.06 KB | Save Response

Pretty Table

```
1 {
2   "data": {
3     "consultas": [
4       {
5         "id": "1",
6         "dataHora": "2026-09-15T14:30",
7         "status": "AGENDADA",
8         "observacoes": "Primeira consulta",
9         "paciente": {
10          "id": "2",
11          "nome": "José Oliveira"
12        },
13        "medico": {
14          "id": "1",
15          "nome": "Dr. João Silva"
```

Perfil Paciente só visualiza suas próprias consultas.

GraphQL > Consultas > Lista de consultas futuras Copy

http://localhost:8080/graphql

Query

Docs Query Authorization Headers Schema Scripts Cookies

Search fields

paciente Usuario!

- ☒ id ID!
- ☐ username String!
- ☒ nome String!
- ☐ email String!

Variables

Body Headers Test

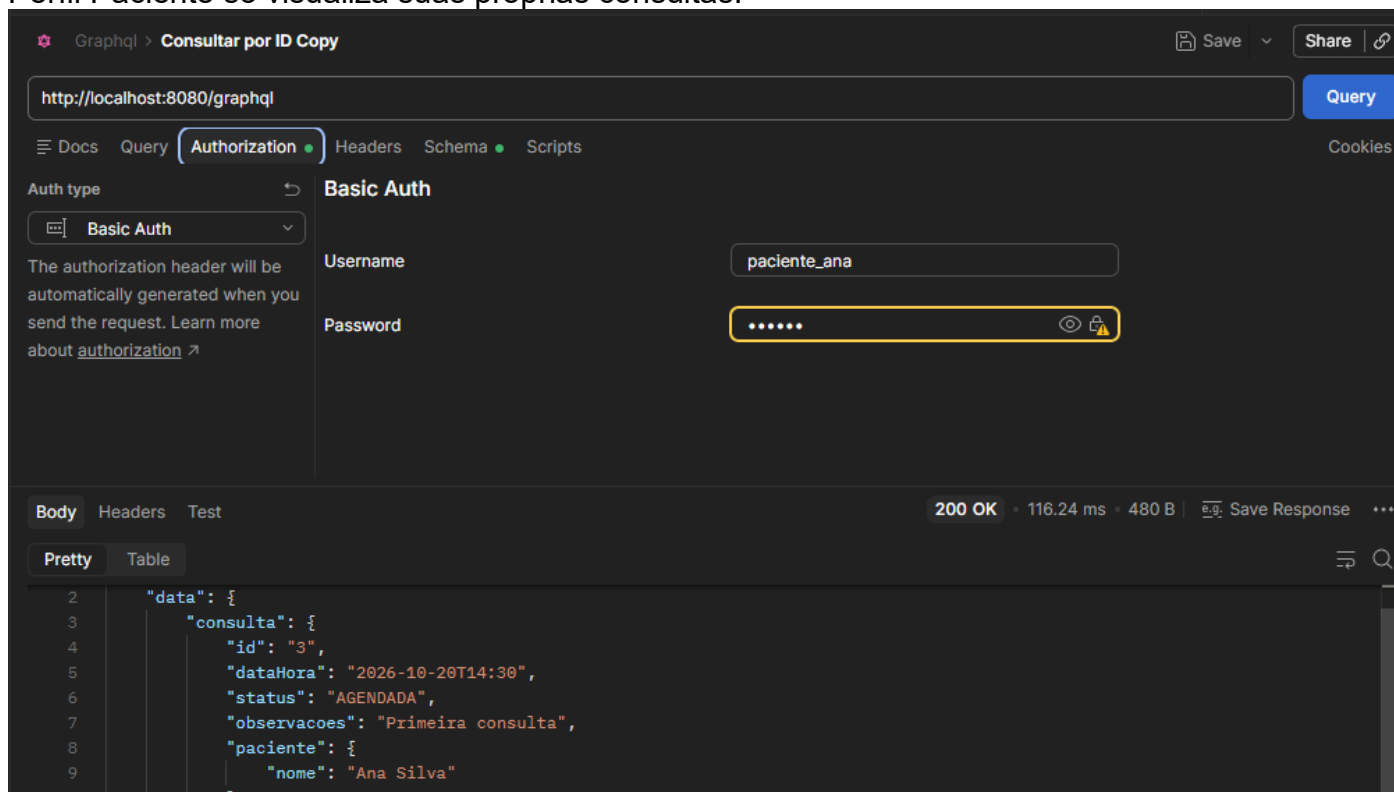
200 OK • 94.78 ms • 671 B | Save Response

Pretty Table

```
9   "paciente": {
10     "id": "3",
11     "nome": "Ana Silva",
12     "role": "PACIENTE"
13   },
14 },
15 {
16   "id": "3",
17   "dataHora": "2026-10-20T14:30",
18   "status": "AGENDADA",
19   "observacoes": "Primeira consulta",
20   "paciente": {
21     "id": "3",
22     "nome": "Ana Silva",
```

Buscar Consulta por ID

Descrição: Os perfis Medico e Enfermeiro tem permissão para visualizar todas as consultas. Perfil Paciente só visualiza suas próprias consultas.



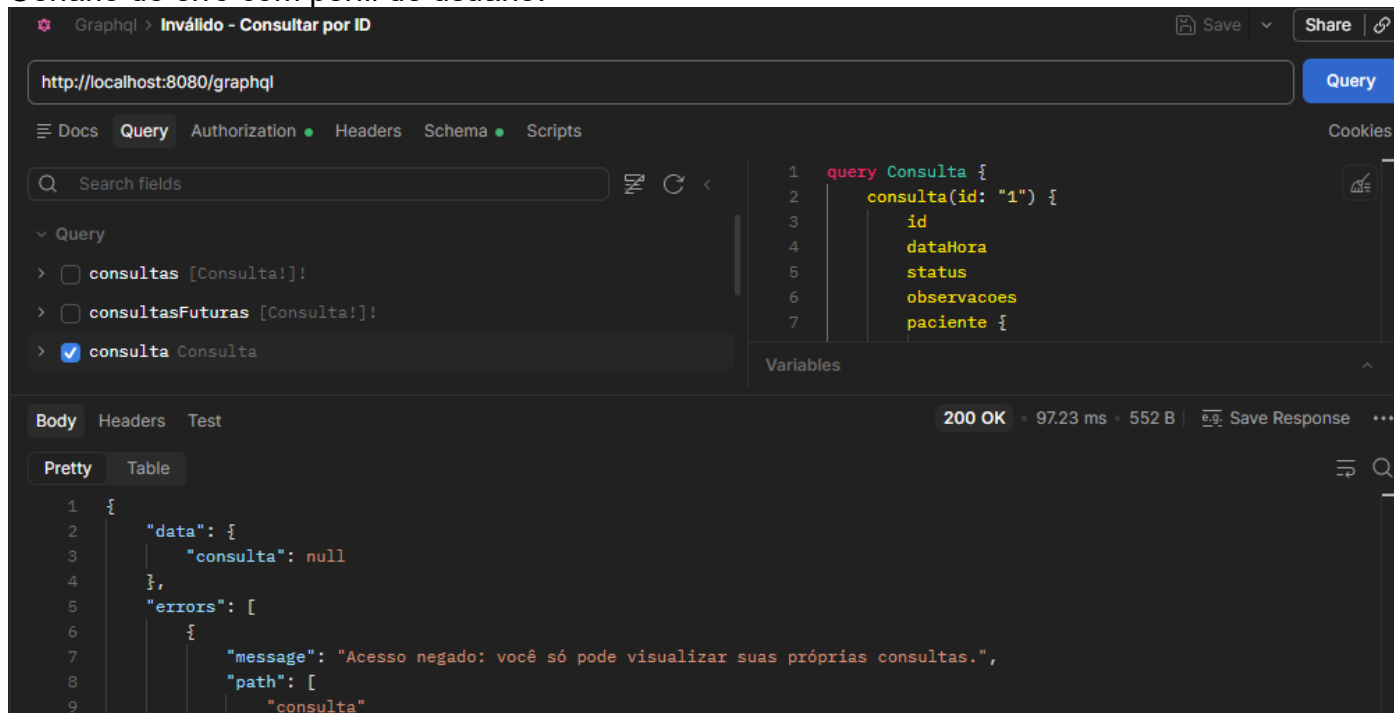
The screenshot shows the GraphQL Playground interface. The URL is `http://localhost:8080/graphql`. The 'Authorization' tab is selected, showing 'Basic Auth' with the username `paciente_ana` and a masked password. The 'Query' tab is also visible. The response is displayed in the 'Body' tab, showing a JSON object with a consultation for 'Ana Silva'.

```
query {
  consulta(id: "3") {
    id
    dataHora
    status
    observacoes
    paciente {
      nome
    }
  }
}
```

Response: `200 OK` • 116.24 ms • 480 B

```
{
  "data": {
    "consulta": {
      "id": "3",
      "dataHora": "2026-10-20T14:30",
      "status": "AGENDADA",
      "observacoes": "Primeira consulta",
      "paciente": {
        "nome": "Ana Silva"
      }
    }
  }
}
```

Cenário de erro com perfil de usuário:



The screenshot shows the GraphQL Playground interface. The URL is `http://localhost:8080/graphql`. The 'Query' tab is selected, showing a query for a consultation with ID '1'. The response is displayed in the 'Body' tab, showing a JSON object with an error message.

```
query Consulta {
  consulta(id: "1") {
    id
    dataHora
    status
    observacoes
    paciente {
      nome
    }
  }
}
```

Response: `200 OK` • 97.23 ms • 552 B

```
{
  "data": {
    "consulta": null
  },
  "errors": [
    {
      "message": "Acesso negado: você só pode visualizar suas próprias consultas.",
      "path": [
        "consulta"
      ]
    }
  ]
}
```

Editar Consulta

Descrição: Os perfis Medico e Enfermeiro tem permissão editar todas as consultas, e o perfil Paciente sem permissão.

GraphQL > **Editar consulta** Save Share

<http://localhost:8080/graphql> Query

Docs **Query** Authorization Headers Schema Scripts Cookies

Search fields

☒ **editarConsulta** Consulta!

☒ **id** ID! ARG

☒ **input** ConsultaInput! ARG

☒ **pacienteId** ID! ARG

☒ **medicoId** ID! ARG

☒ **dataHora** String! ARG

```
1 mutation EditarConsulta {
2   editarConsulta(
3     id: "2"
4     input: {
5       pacienteId: "2"
6       medicoId: "1"
7       dataHora: "2026-09-20T14:30"
8       observacoes: "Alteração de consulta"
9     }
10  ) { }
```

Variables

Body Headers Test **200 OK** • 93.19 ms • 497 B Save Response

Pretty Table

```
1 {
2   "data": {
3     "editarConsulta": {
4       "id": "2",
5       "dataHora": "2026-09-20T14:30",
6       "status": "AGENDADA",
7       "observacoes": "Alteração de consulta",
8       "paciente": {
9         "nome": "José Oliveira"
10      }
11    }
12  }
```

Cenário de erro:

GraphQL > **Editar consulta Copy** Save Share

<http://localhost:8080/graphql> Query

Docs **Query** Authorization Headers Schema Scripts Cookies

Search fields

☐ **consultarStatus** Consulta!

☐ **consulta** Consulta

Mutation

☐ **criarConsulta** Consulta!

☒ **editarConsulta** Consulta!

☐ **cancelarConsulta** Consulta!

```
1 mutation EditarConsulta {
2   editarConsulta(
3     id: "2"
4     input: {
5       pacienteId: "2"
6       medicoId: "1"
7       dataHora: "2026-09-20T14:30"
8       observacoes: "Alteração de consulta"
9     }
10  ) { }
```

Variables

Body Headers Test **200 OK** • 95.34 ms • 544 B Save Response

Pretty Table

```
1 {
2   "data": null,
3   "errors": [
4     {
5       "message": "Acesso negado. Você não tem permissão para executar esta ação.",
6       "path": [
7         "editarConsulta"
8       ]
9     }
10  ]
11 }
```

```
agendamento-app | 2026-09-13T00:00:59.999Z INFO 1 --- [service-agendamento] [nio-8080-exec-4] b.c.s.a.service.ConsultaService : Editando consulta ID =2
agendamento-app | 2026-09-13T00:01:00.065Z INFO 1 --- [service-agendamento] [nio-8080-exec-4] b.c.s.a.producer.ConsultaProducer : 🚧 Publicando evento
| Consulta ID: 2 | Ação: EDITADA | Paciente: José Oliveira | Data: 20/09/2026 14:30 | RoutingKey: consulta.editada
agendamento-app | 2026-09-13T00:01:00.067Z INFO 1 --- [service-agendamento] [nio-8080-exec-4] b.c.s.a.producer.ConsultaProducer : ✅ Evento publicado com sucesso | Consulta ID: 2
notificacao-app | 2026-09-13T00:01:00.074Z INFO 1 --- [service-notificacao] [ntContainer#0-4] b.c.s.n.consumer.NotificacaoConsumer : 📬 Evento recebido | Consulta ID: 2 | Ação: EDITADA
notificacao-app | 2026-09-13T00:01:00.076Z INFO 1 --- [service-notificacao] [ntContainer#0-4] b.c.s.n.consumer.NotificacaoConsumer : 
notificacao-app | LEMBRETE DE CONSULTA - EDITADA
notificacao-app | Paciente : José Oliveira
notificacao-app | E-mail : jose@email.com
notificacao-app | Médico : Dr. João Silva
notificacao-app | Data : 20/09/2026 às 14:30
notificacao-app | Status : AGENDADA
```

Descrição: Os perfis Medico e Enfermeiro tem permissão cancelar todas as consultas, e o perfil Paciente sem permissão.

The screenshot shows the GraphQL Playground interface. At the top, the URL is `http://localhost:8080/graphql`. The query editor contains the following query:

```
mutation CancelarConsulta {
  cancelarConsulta(id: "1") {
    id
    dataHora
    status
    observacoes
    paciente {
      id
      nome
    }
  }
}
```

The query is executed, and the response is shown in the "Body" tab. The response is a JSON object:

```
{
  "data": {
    "cancelarConsulta": {
      "id": "1",
      "dataHora": "2026-09-15T14:30",
      "status": "CANCELADA",
      "observacoes": "Primeira consulta",
      "paciente": {
        "id": "2",
        "nome": "José Oliveira"
      }
    },
    "medico": {
      "id": "1",
      "nome": "Dr. João Silva"
    }
  }
}
```

Cenário de erro:

The screenshot shows the GraphQL Playground interface. The URL is `http://localhost:8080/graphql`. The query is `mutation CancelarConsulta { cancelarConsulta(id: "2") { id dataHora status observacoes paciente { } } }`. The response is a 200 OK status with a message: `"message": "Acesso negado. Você não tem permissão para executar esta ação.", "path": [ "cancelarConsulta" ]`.

```
mutation CancelarConsulta {
  cancelarConsulta(id: "2") {
    id
    dataHora
    status
    observacoes
    paciente {
    }
  }
}
```

```
{
  "message": "Acesso negado. Você não tem permissão para executar esta ação.",
  "path": [
    "cancelarConsulta"
  ]
}
```

Alterar status da consulta para realizada

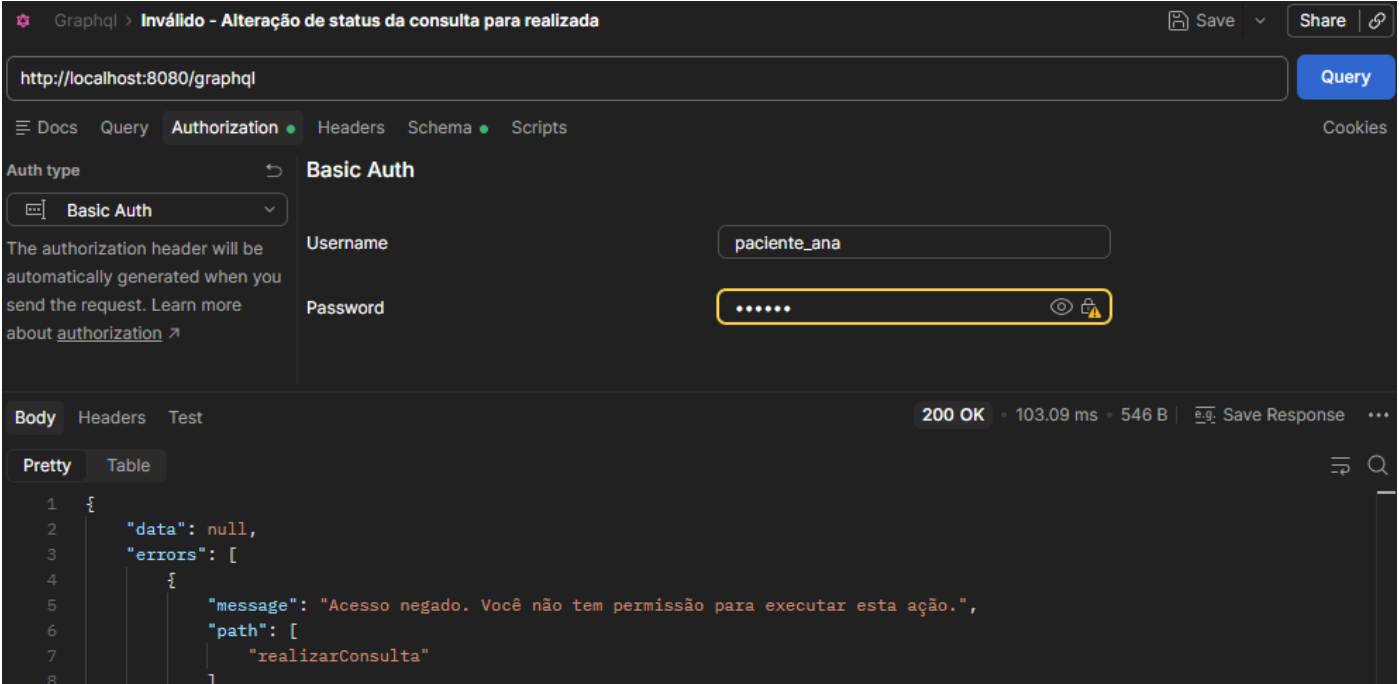
Descrição: Os perfis Medico e Enfermeiro tem permissão alterar os status de consulta agendada para realizada, e o perfil Paciente sem permissão.

The screenshot shows the GraphQL Playground interface. The URL is `http://localhost:8080/graphql`. The query is `mutation RealizarConsulta { realizarConsulta(id: "2") { id dataHora status observacoes paciente { nome } } }`. The response is a 200 OK status with a message: `"data": { "realizarConsulta": { "id": "2", "dataHora": "2026-09-20T14:30", "status": "REALIZADA", "observacoes": "Alteração de consulta", "paciente": { "nome": "José Oliveira" } } }`.

```
mutation RealizarConsulta {
  realizarConsulta(id: "2") {
    id
    dataHora
    status
    observacoes
    paciente {
      nome
    }
  }
}
```

```
{
  "data": {
    "realizarConsulta": {
      "id": "2",
      "dataHora": "2026-09-20T14:30",
      "status": "REALIZADA",
      "observacoes": "Alteração de consulta",
      "paciente": {
        "nome": "José Oliveira"
      }
    }
  }
}
```

Cenário de erro:



11.3 Ferramentas

- **Postman** — para as requisições REST e GraphQL;
- **Docker Compose** — para ambiente reproduzível;
- **Logs** — para validação da comunicação assíncrona.

12. Decisões Arquiteturais e Justificativas

12.1 ADR-001 — Uso de GraphQL em vez de REST para consultas

Campo	Descrição
Contexto	Necessidade de consultas flexíveis sobre o histórico de consultas
Decisão	Utilizar GraphQL como camada de consulta
Justificativa	Elimina overfetching/underfetching, permite evolução contínua do schema e atende ao requisito do enunciado
Consequências	Curva de aprendizado maior; necessidade de ferramenta de teste adequada (GraphiQL/Postman)

12.2 ADR-002 — Uso de RabbitMQ em vez de Kafka

Campo	Descrição
-------	-----------

Contexto	Necessidade de comunicação assíncrona entre serviços
Decisão	Utilizar RabbitMQ
Justificativa	Modelo de filas simples e roteamento flexível; broker já consolidado; menor complexidade operacional que Kafka para o escopo
Consequências	Menor throughput que Kafka (não é limitante para o cenário)

12.3 ADR-003 — Uso de Basic Auth em vez de JWT

Campo	Descrição
Contexto	Necessidade de autenticação simples e aderente ao enunciado
Decisão	Utilizar Basic Auth
Justificativa	Atende literalmente ao requisito "autenticação básica"; reduz complexidade; adequado ao escopo acadêmico
Consequências	Cada requisição envia credenciais; recomendável uso de HTTPS em produção

12.4 ADR-004 — Modelo único de Usuario com enum Role

Campo	Descrição
Contexto	Três perfis com atributos comuns
Decisão	Uma única entidade com discriminador de perfil
Justificativa	Simplicidade; sem atributos específicos exigidos pelo enunciado
Consequências	Se surgirem atributos muito distintos, será necessário refatorar para herança JPA

12.5 ADR-005 — Uso de record para DTOs

Campo	Descrição
Contexto	Necessidade de objetos imutáveis para transferência de dados
Decisão	Utilizar record (Java 21)

Justificativa	Elimina boilerplate; imutabilidade intrínseca; suporte nativo a serialização JSON
Consequências	Não compatível com frameworks que exigem construtor sem argumentos (não é o caso)

13. Considerações Finais

13.1 Aderência aos Requisitos

Requisito	Status	Evidência
Autenticação com Spring Security	✓ Atendido	SecurityConfig + CustomUserDetailsService
Níveis de acesso (Médico, Enfermeiro, Paciente)	✓ Atendido	@PreAuthorize + filtros no Service
GraphQL para consultas flexíveis	✓ Atendido	schema.graphqls + ConsultaResolver
Serviço de Agendamento	✓ Atendido	service-agendamento
Serviço de Notificações	✓ Atendido	service-notificacao
Comunicação assíncrona com RabbitMQ	✓ Atendido	ConsultaProducer + NotificacaoConsumer
Docker Compose	✓ Atendido	docker-compose.yml

13.2 Aprendizados

O desenvolvimento deste projeto consolidou conhecimentos em:

1. **Arquitetura de microserviços** com separação por domínio;
2. **Spring Security** aplicado a cenários reais de RBAC;
3. **GraphQL** como alternativa ao REST em cenários de consultas complexas;
4. **Mensageria assíncrona** com RabbitMQ e AMQP;
5. **Containerização** com Docker e Docker Compose;
6. **Boas práticas** de código (DTOs com record, camadas bem definidas, injeção de dependência por construtor).

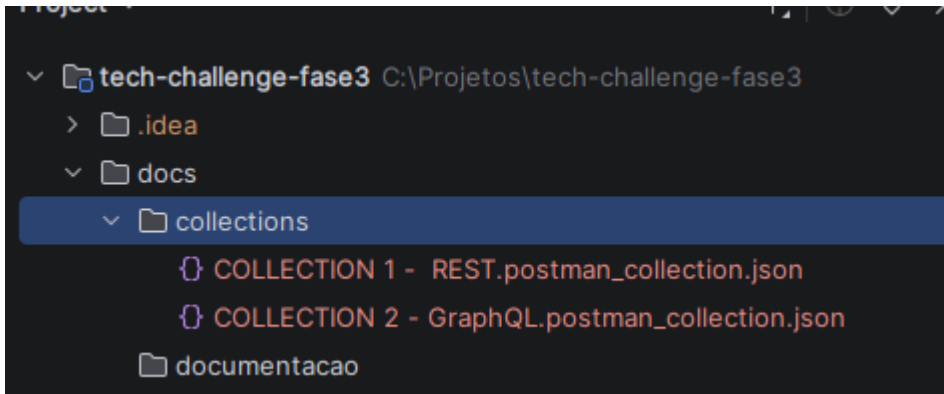
14. Anexos

Anexo A — Repositório do Projeto

<https://github.com/Pamt-o/tech-challenge-fase3>

Anexo B — Collections Postman e Documentação

Arquivos disponível na raiz do repositório na pasta docs consta as Collections e a Documentação:



Anexo C — Comandos de Execução

bash

Clonar

`git clone https://github.com/Pamt-o/tech-challenge-fase3`

`cd tech-challenge-fase3`

Subir

`docker-compose up -d --build`

Verificar

`docker-compose ps`

`docker-compose logs -f`